# Customer Shopping Behavior Analysis

## 1. Project Overview

This project analyzes customer shopping behavior using transactional data from 3,900 purchases across various product categories. The goal is to uncover insights into spending patterns, customer segments, product preferences, and subscription behavior to guide strategic business decisions.

## 2. Dataset Summary

- Rows: 3,900

- Columns: 18

- Key Features:

    - Customer demographics (Age, Gender, Location, Subscription Status)

    - Purchase details (Item Purchased, Category, Purchase Amount, Season, Size, Color)

    - Shopping behavior (Discount Applied, Promo Code Used, Previous Purchases, Frequency of Purchases, Review Rating, Shipping Type)

    - Missing Data: 37 values in Review Rating column

## 3. Exploratory Data Analysis using Python

We began with data preparation and cleaning in Python:

- Data Loading: Imported the dataset using pandas.
- Initial Exploration: Used df.info() to check structure and .describe() for summary statistics

| | Customer ID | Age | Gender | Item Purchased | Category | Purchase Amount (USD) | Location | Size | Color | Season | Review Rating | Subscription Status | Shipping Type | Discount Applied | Promo Code Used |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **count** | 3900.000000 | 3900.000000 | 3900 | 3900 | 3900 | 3900.000000 | 3900 | 3900 | 3900 | 3900 | 3863.000000 | 3900 | 3900 | 3900 | 3900 |
| **unique** | NaN | NaN | 2 | 25 | 4 | NaN | 50 | 4 | 25 | 4 | NaN | 2 | 6 | 2 | 2 |
| **top** | NaN | NaN | Male | Blouse | Clothing | NaN | Montana | M | Olive | Spring | NaN | No | Free Shipping | No | No |
| **freq** | NaN | NaN | 2652 | 171 | 1737 | NaN | 96 | 1755 | 177 | 999 | NaN | 2847 | 675 | 2223 | 2223 |
| **mean** | 1950.500000 | 44.068462 | NaN | NaN | NaN | 59.764359 | NaN | NaN | NaN | NaN | 3.750065 | NaN | NaN | NaN | NaN |
| **std** | 1125.977353 | 15.207589 | NaN | NaN | NaN | 23.685392 | NaN | NaN | NaN | NaN | 0.716983 | NaN | NaN | NaN | NaN |
| **min** | 1.000000 | 18.000000 | NaN | NaN | NaN | 20.000000 | NaN | NaN | NaN | NaN | 2.500000 | NaN | NaN | NaN | NaN |
| **25%** | 975.750000 | 31.000000 | NaN | NaN | NaN | 39.000000 | NaN | NaN | NaN | NaN | 3.100000 | NaN | NaN | NaN | NaN |
| **50%** | 1950.500000 | 44.000000 | NaN | NaN | NaN | 60.000000 | NaN | NaN | NaN | NaN | 3.800000 | NaN | NaN | NaN | NaN |
| **75%** | 2925.250000 | 57.000000 | NaN | NaN | NaN | 81.000000 | NaN | NaN | NaN | NaN | 4.400000 | NaN | NaN | NaN | NaN |
| **max** | 3900.000000 | 70.000000 | NaN | NaN | NaN | 100.000000 | NaN | NaN | NaN | NaN | 5.000000 | NaN | NaN | NaN | NaN |

Activate Windows
Go to Settings to activate W

| Previous Purchases | Payment Method | Frequency of Purchases |
|---|---|---|
| 3900.000000 | 3900 | 3900 |
| NaN | 6 | 7 |
| NaN | PayPal | Every 3 Months |
| NaN | 677 | 584 |
| 25.351538 | NaN | NaN |
| 14.447125 | NaN | NaN |
| 1.000000 | NaN | NaN |
| 13.000000 | NaN | NaN |
| 25.000000 | NaN | NaN |
| 38.000000 | NaN | NaN |
| 50.000000 | NaN | NaN |

● Missing Data Handling: Checked for null values and imputed missing values in the Review Rating column using the median rating of each product category.

● Column Standardization: Renamed columns to snake case for better readability and documentation.

● Feature Engineering:

- ○ Created age_group column by binning customer ages.
- ○ Created purchase_frequency_days column from purchase data.

● Data Consistency Check: Verified if discount_applied and promo_code_used were redundant; dropped promo_code_used.

● Database Integration: Connected Python script to PostgreSQL and loaded the cleaned DataFrame into the database for SQL analysis.

## 4. Data Analysis using SQL (Business Transactions)

We performed structured analysis in PostgreSQL to answer key business questions:

**1. Revenue by Gender** – Compared total revenue generated by male vs. female customers.

```
select gender, sum(purchase_amount) as revenue
from customer
group by gender
```

| | gender text | revenue numeric |
|---|---|---|
| 1 | Female | 75191 |
| 2 | Male | 157890 |

**2. High-Spending Discount Users** – Identified customers who used discounts but still spent above the average purchase amount.

```
select customer_id, purchase_amount
from customer
where discount_applied = 'Yes' and purchase_amount >= (select avg(purchase_amount) from customer)
```

| | customer_id bigint | purchase_amount bigint |
|---|---|---|
| 1 | 2 | 64 |
| 2 | 3 | 73 |
| 3 | 4 | 90 |
| 4 | 7 | 85 |
| 5 | 9 | 97 |
| 6 | 12 | 68 |
| 7 | 13 | 72 |
| 8 | 16 | 81 |
| 9 | 20 | 90 |
| 10 | 22 | 62 |
| 11 | 24 | 88 |
| 12 | 29 | 94 |
| 13 | 32 | 79 |
| 14 | 33 | 67 |
| 15 | 35 | 91 |
| 16 | 37 | 69 |
| 17 | 40 | 60 |
| 18 | 41 | 76 |
| Total rows: 839 of 839 | | Query complete |

**3. Top 5 Products by Rating** – Found products with the highest average review ratings.

```
select item_purchased, round(avg(review_rating::numeric),2) as "Average Product Rtaing"
from customer
group by item_purchased
order by avg(review_rating) desc
limit 5
```

| | item_purchased text | Average Product Rtaing numeric |
|---|---|---|
| 1 | Gloves | 3.86 |
| 2 | Sandals | 3.84 |
| 3 | Boots | 3.82 |
| 4 | Hat | 3.80 |
| 5 | Skirt | 3.78 |

**4. Shipping Type Comparison** – Compared average purchase amounts between Standard and Express shipping.

```
select shipping_type,
round(avg(purchase_amount),2)
from customer
where shipping_type in ('Standard', 'Express')
group by shipping_type
```

| | shipping_type text | round numeric |
|---|---|---|
| 1 | Standard | 58.46 |
| 2 | Express | 60.48 |

**5. Subscribers vs. Non-Subscribers** – Compared average spend and total revenue across subscription status.

```
select subscription_status,
count(customer_id) as total_customers,
round(avg(purchase_amount),2) as avg_spend,
round(sum(purchase_amount),2) as total_revenue
from customer
group by subscription_status
order by total_revenue,avg_spend desc;
```

Data Output Messages Notifications

| | subscription_status text | total_customers bigint | avg_spend numeric | total_revenue numeric |
|---|---|---|---|---|
| 1 | Yes | 1053 | 59.49 | 62645.00 |
| 2 | No | 2847 | 59.87 | 170436.00 |

**6. Discount-Dependent Products** – Identified 5 products with the highest percentage of discounted purchases.

```
select item_purchased,
round(100 * sum(case when discount_applied ='Yes' then 1 else 0 end)/count(*) , 2) as discount_rate
from customer
group by item_purchased
order by discount_rate desc
limit 5;
```

| | item_purchased text | discount_rate numeric |
|---|---|---|
| 1 | Hat | 50.00 |
| 2 | Sneakers | 49.00 |
| 3 | Coat | 49.00 |
| 4 | Sweater | 48.00 |
| 5 | Pants | 47.00 |

**7. Customer Segmentation** – Classified customers into New, Returning, and Loyal segments based on purchase history.

```
with customer_type as (
select customer_id, previous_purchases,
    case
        when previous_purchases = 1 then 'New'
        when previous_purchases between 2 and 10 then 'Returning'
        else 'Loyal'
        end as customer_segment
    from customer
)

select customer_segment,count(*) as "Numaber of Customers"
from customer_type
group by customer_segment
```

Data Output Messages Notifications

| | customer_segment text | Numaber of Customers bigint |
|---|---|---|
| 1 | Loyal | 3116 |
| 2 | New | 83 |
| 3 | Returning | 701 |

**8. Top 3 Products per Category** – Listed the most purchased products within each category.

```
with item_counts as (
select category,
item_purchased,
    count(customer_id) as total_orders,
    row_number() over(partition by category order by count(customer_id) desc ) as item_rank
from customer
group by category, item_purchased
)

select item_rank,category, item_purchased, total_orders
from item_counts
where item_rank <= 3
```

| | item_rank bigint | category text | item_purchased text | total_orders bigint |
|---|---|---|---|---|
| 1 | 1 | Accessories | Jewelry | 171 |
| 2 | 2 | Accessories | Sunglasses | 161 |
| 3 | 3 | Accessories | Belt | 161 |
| 4 | 1 | Clothing | Blouse | 171 |
| 5 | 2 | Clothing | Pants | 171 |
| 6 | 3 | Clothing | Shirt | 169 |
| 7 | 1 | Footwear | Sandals | 160 |
| 8 | 2 | Footwear | Shoes | 150 |
| 9 | 3 | Footwear | Sneakers | 145 |
| 10 | 1 | Outerwear | Jacket | 163 |
| 11 | 2 | Outerwear | Coat | 161 |

**9. Repeat Buyers & Subscriptions** – Checked whether customers with >5 purchases are more likely to subscribe.

```
select subscription_status,
count(customer_id) as repeat_buyers
from customer
where previous_purchases > 5
group by subscription_status
```

Data Output Messages Notifications

| | subscription_status text | repeat_buyers bigint |
|---|---|---|
| 1 | No | 2518 |
| 2 | Yes | 958 |

**10. Revenue by Age Group** – Calculated total revenue contribution of each age group.

```
select age_group,
sum(purchase_amount) as total_revenue
from customer
group by age_group
order by total_revenue desc
```

Data Output  Messages  Notifications

| | age_group text | total_revenue numeric |
|---|---|---|
| 1 | Young Adult | 62143 |
| 2 | Middle-aged | 59197 |
| 3 | Adult | 55978 |
| 4 | Senior | 55763 |

## 5. Dashboard in Power BI

Finally, we built an interactive dashboard in Power BI to present insights visually.

## 6. Business Recommendations

- Boost Subscriptions – Promote exclusive benefits for subscribers.
- Customer Loyalty Programs – Reward repeat buyers to move them into the “Loyal” segment.

- Review Discount Policy – Balance sales boosts with margin control.
- Product Positioning – Highlight top-rated and best-selling products in campaigns.
- Targeted Marketing – Focus efforts on high-revenue age groups and express-shipping users.